Hiragana Writing Buddy — Product Plan

Oct 13, 2026 · @Liz

At a glance

Hiragana Buddy is a free web app that teaches 5–6-year-olds in English/Japanese immersion kindergarten to write their name in hiragana with correct stroke order. As product lead, I took it from a problem statement to a live, tested app in five days (Oct 8–13, 2026). I designed it with Claude and directed Claude Code to build it to my spec. Try it at hiraganabuddy.pages.dev.

- **My role:** product lead. I framed the problem, wrote the design doc and agent brief, and set the quality bars. I also made every UX call, directed the art and voice, and ran the releases.
- **Speed:** about 10 hours of hands-on time over five days. The first testable build came in the first session, and the app went live on Cloudflare Pages on Oct 13 after 3 reviewed pull requests and 40+ rounds of directed feedback.
- **Product:** a grown-up types or says a name, the child sees it as hiragana tiles, then learns each letter in three steps: follow me, trace, write it yourself. Order is enforced, mistakes are never punished, and everything stays on the device.
- **Quality:** 76 automated tests pass (10 matcher, 24 unit, 42 browser), and drawing holds 50–60 fps under 4× CPU throttling.
- **Early signal:** “This app is amazing and super cute.” — Parent of a kindergartner

The problem

Young immersion students are asked to write hiragana before they can read, and paper worksheets can’t tell them when a stroke goes the wrong way. The challenge is to build sound familiarity and correct stroke order together, at an age when a child can’t read the instructions. I set out to turn classroom hiragana assignments into a self-correcting digital experience. It needed immediate feedback and enough delight that children want to come back.

Who	What they need	What gets in the way today
Child, 5–6, pre-reader	To hear each sound, see how it is drawn, and know right away that they did it	Can't read prompts; developing motor control; discouraged by red marks
Parent	A five-minute way to practise the class's words at home	Often can't read kana, so can't check a worksheet
Teacher	Practice that reinforces the stroke order taught in class	No visibility into home practice; worksheets give no feedback

Three constraints shaped everything else:

- **Names come first, and they are hard.** English names have no single correct hiragana spelling, and automatic conversion is wrong for about 1 in 4 names (Noah → ノー). A grown-up must always be able to check and fix the tiles.
- **The audience can't read.** Every screen after word entry has to work by picture, sound and gesture alone.
- **These are children.** No accounts, no tracking, no network calls, so there is no consent flow and no children's-privacy exposure.

Product principles

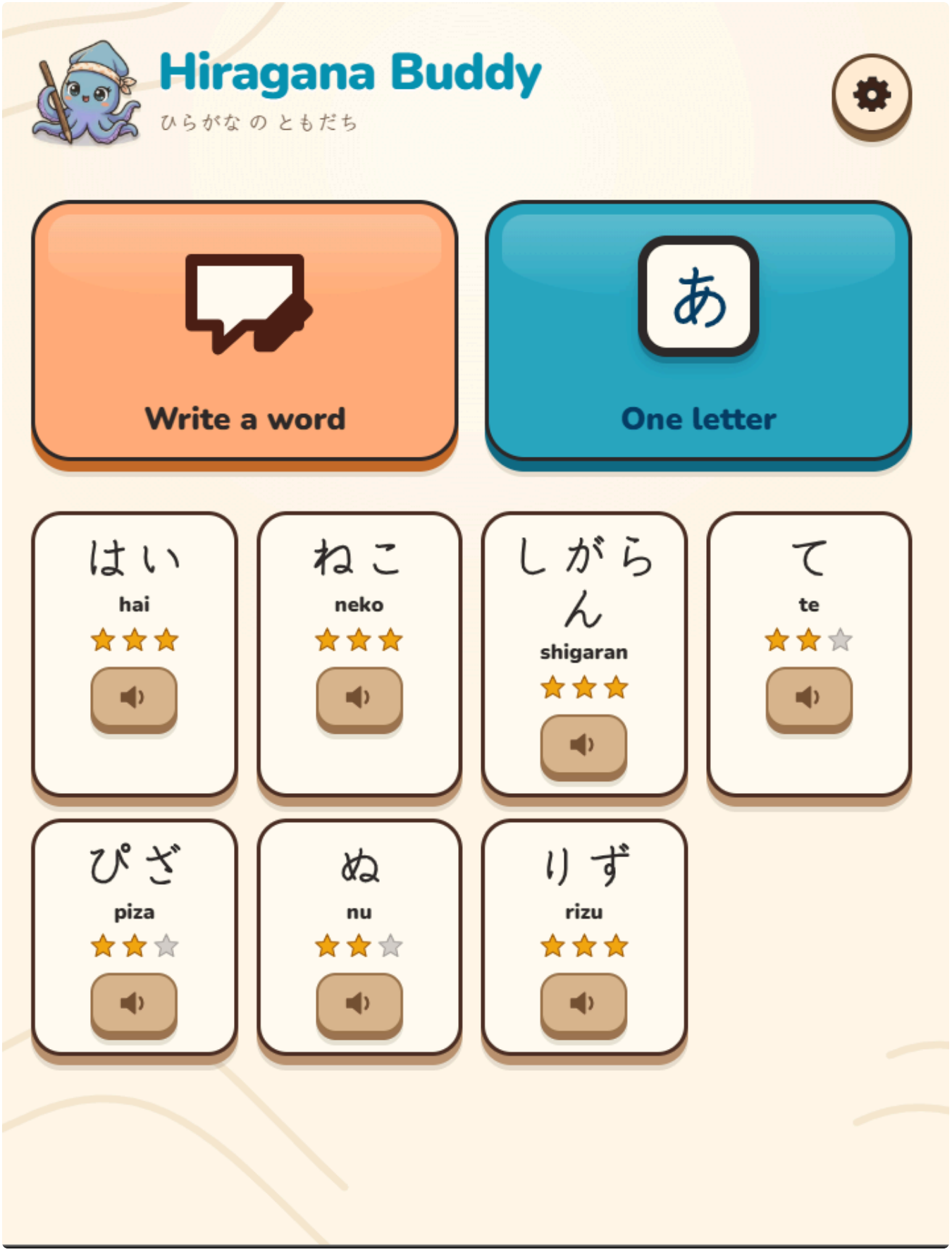
I wrote six non-negotiables into the agent brief before any code existed, so every build decision could be checked against them.

Principle	The rule	Why
Never discouraging	No red, no X, no buzzer, no timers, no lives; every finished word earns at least 1 star	A five-year-old who feels they failed stops practising
Order is the lesson	Only the next expected stroke is accepted; a later stroke gets "That's stroke 3! Let's do stroke 2 first"	Stroke order is what worksheets can't enforce

Principle	The rule	Why
Generous on wobble	Shape and start point are judged with wide tolerances that tighten across the three steps	Motor control is still developing; strictness belongs in order, not neatness
Ink under the finger, now	Ink draws on the next frame; 40 fps floor, 60 target, on a mid-tier phone	Lag breaks the link between hand and letter
Palms do nothing	Touches outside the cell, large contacts and second fingers are ignored; a live stroke is never cancelled	Children rest their hands on tablets
Light and quick to load	Under 150 KB of code on first load; voice clips about 5 KB each, fetched only when needed and cached for offline; heavy models only loaded on demand	Families open it on whatever phone is at hand, often on mobile data; a slow first load loses a five-year-old
Nothing leaves the device	Static site, no accounts, no analytics, works offline	Privacy by design for children; near-zero cost to share with families

The experience

A grown-up enters the word once; from then on the child needs only to look, listen and draw.


Hiragana Buddy
ひらがな の ともだち
Write a word
あ
One letter
はい
hai
ねこ
neko
しがらん
shigaran
て
te
ぴざ
piza
ぬ
nu
りず
rizu


お
は
よ
3

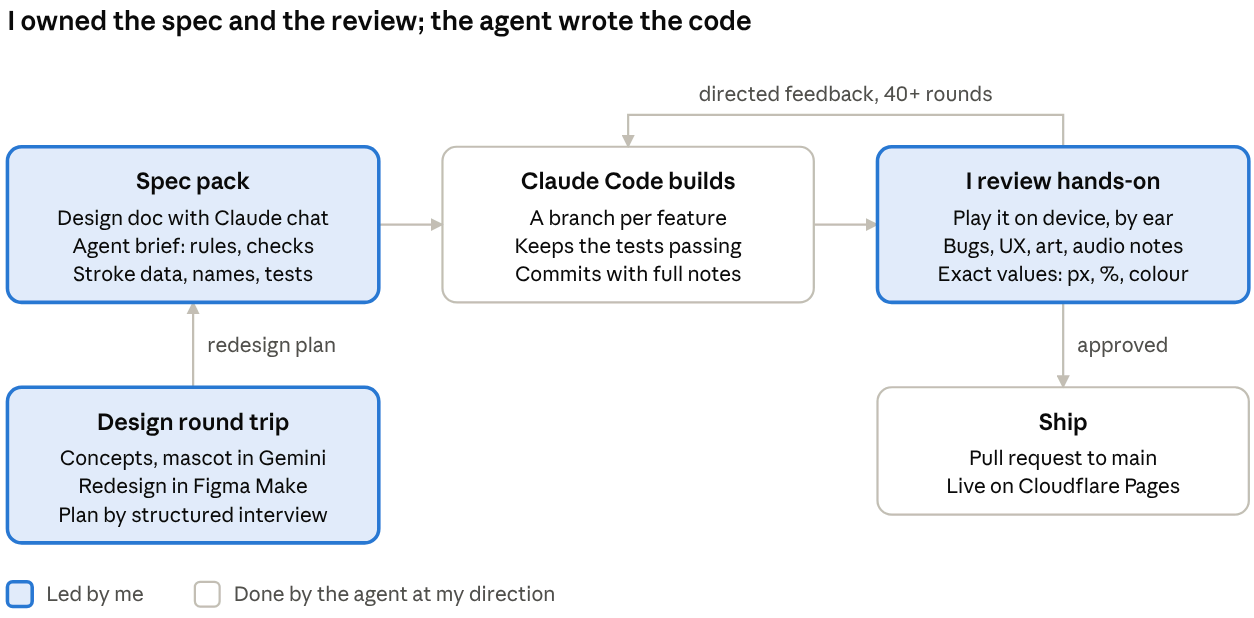

おはよ
Home

1. **Enter a word.** Type a name in English (Emma), type how it sounds (ne-ko), or on Android and Windows say it into the private, on-device microphone. A curated list of 176 names and 44 words gives the usual Japanese spelling. Anything else is spelled by sound automatically, with no mode switch to learn.
2. **Check the tiles.** Each kana appears as a tile that speaks when tapped, so a parent who can't read kana can still check by ear and swap a tile.
3. **Follow me.** The octopus draws stroke 1, and the child traces it. Then stroke 2, and so on.
4. **Trace.** The whole letter shows as a faint ghost with numbered start dots.
5. **Write it yourself.** An empty square with dashed guide lines. This attempt is scored.
6. **My word.** The child's own letters appear side by side, with stars, a dance and the word spoken aloud. The session is saved to the practice shelf.

When a stroke misses, the ink fades softly and the right stroke replays. After three misses the app drops back to "follow me" for that stroke and lets the child finish together with it.

How I built it

I treated the AI agent as an engineering team that needs a clear spec, firm guardrails and fast, specific review. Before any code, I prepared a resource pack. It held the design doc, an agent brief of rules and acceptance checks, stroke data for 87 glyphs, a seed list of 220 names and words, and a reference stroke matcher with tests.



build loop · spec, agent build, hands-on review

The speed came from the review loop. I tested every build by hand and by ear, then returned precise notes like “move it up 8 points” and “the trace guide 10% darker”. For bigger changes I designed outside the code first, and ran a structured interview so the agent had one clear plan before it built anything.

UI and UX design

The look came from an AI-centred pipeline with three tools, each used for what it does best. Google Gemini explored concepts, palette and the mascot. Figma Make turned them into exact tokens and a working layout. Claude Code then applied that system to every screen.

Stage	Tool	What I directed	Output
Explore	Google Gemini	Concepts for layout, palette, theming and a mascot; decluttering and affordance passes; character selection and animation polish	Concept screens, a rough palette, mascot key art on pure white, a 16-frame dance sheet
Specify	Figma Make	A home page built from the Gemini references: phone first, then tablet portrait and landscape; reusable cards and buttons; a shared press effect; reviewed against the references	Exact design tokens, a responsive layout and a working mockup in React, TypeScript and Tailwind
Apply	Claude Code	The mockup’s style ported to every screen from a plan made in a structured interview; art cut out and frame-aligned in code	The shipped UI (PR #1–#3)

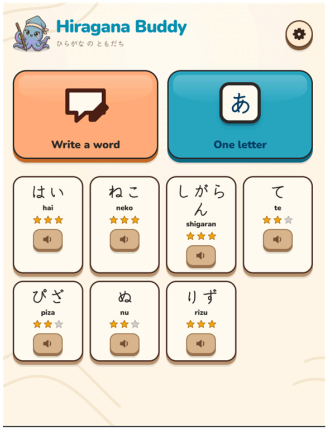
1 First build



2 Gemini concept



3 Shipped



1 Eight mascot concepts



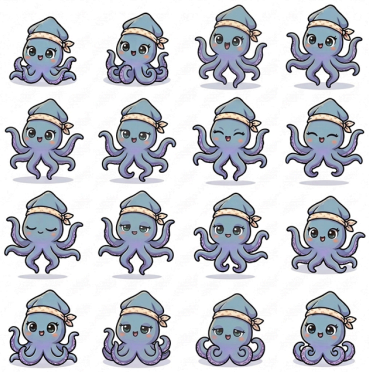
2 Squid variations



3 Final: Ika-Ika



4 16-frame dance sheet



The palette pairs warm neutrals for surfaces and outlines with seven brights for ink, guides and celebrations, chosen from Tailwind CSS:

- **Repeatable and harmonious:** named Tailwind steps carry straight into code, and shades from shared scales sit well together.
- **Accessible:** contrast between steps is predictable, so text and controls meet contrast standards while the range stays charming and kid-friendly.

- **Custom where Tailwind stops:** the tan and brown fill out the neutrals with desaturated amber shades that Tailwind doesn't offer.



The UX calls behind the look:

- **One clear target per job.** Practice cards became uniform squares. Tapping the card face practises, and a sound button sits inside the card boundary, in a low-contrast tan so it doesn't pull a child away from practising.
- **Less on screen.** Corner decorations gave way to a single round settings gear. Props, pebbles and scenery came out wherever they competed with the letter.
- **A mascot with a reason.** From eight concepts spanning Pacific Northwest wildlife, sandbox mark-makers and Japanese folklore, I chose Ika-Ika the squid. A squid makes ink, which ties it to drawing. Its headband and expressive face suit 5–7-year-olds, and it holds a simple wooden sand stylus instead of brush and ink tools.
- **One palette everywhere.** The mascot's colours match the Tailwind-based UI tokens, and ink colours, stroke guides and star bursts use the palette's brights.
- **Motion that stays light.** The dance is 16 frames on a 4×4 grid, built for step animation with no redraw cost. Frame 16 matches frame 1 for a seamless loop, blinks are worked in, and the ground shadow is anchored so the mascot never jitters.
- **Tactile controls.** Every button moves down and darkens briefly when pressed, like a physical toy.

Zero to deployed in five days

About 10 hours of hands-on time over five working days took the app from an empty repo to a live, redesigned product merged to main.

Date	Hands-on hours	Milestone
Oct 8	1.9	First testable build; recorded Japanese voice; first UX passes
Oct 9	0.4	Figma design export; voice input for word entry (Android and Windows)
Oct 11	0.6	Redesign planned in a structured interview; every screen restyled (PR #1)
Oct 12	3.5	Point-level polish of home, writing and summary (PR #2); sand, clip and spelling fixes
Oct 13	3.7	Voice polish by ear; live on Cloudflare Pages; merged to main (PR #3)
Total	**10.1**	17 commits, 3 reviewed pull requests

Hours run from my first to my last prompt or answer each day. A gap of more than an hour counts as a break and isn't counted.

Key decisions and trade-offs

The pattern behind most of these calls: protect the child's experience and the app's lightness, and cut scope before cutting quality.

Decision	Options weighed	My call	Why
Friendlier Japanese voice	Download neural voices on device (1 GB+); keep robotic browser speech; pre-render clips	Pre-render 400 clips with the Kokoro model, 1.8 MB total; Python stays on my machine only	Kept the app light for families while fixing the voice that sounded "lifeless". I chose the voice (jf_alpha) from a side-by-side of four
English names to kana	Machine-learning model first; curated list plus a grown-up check	A curated list of 176 names and 44 words, with editable tiles; model fallback deferred	In a spot test the model mis-spelled about 1 in 4 names. A wrong name is worse than a manual fix

Decision	Options weighed	My call	Why
Unknown words	A mode toggle for the parent; switch automatically	Switch to spelling by sound automatically	One less concept for a busy parent
Practice order	Whole word per phase; each letter through all three phases	Each letter through all three phases, then the next	Short loops suit a five-year-old's attention
Repeated misses	Keep retrying; lower the bar; go back to the demo	After 3 misses, drop back to "follow me" for that stroke	Recovers without ever showing failure
Script for foreign names	Katakana (the adult norm); hiragana	Hiragana now, katakana as v2	Schools teach hiragana first, and children start with their name
Visual redesign	Iterate in code; design in a tool	Explored concepts and the mascot in Google Gemini, set exact tokens and layout in Figma Make, then planned the port in a structured interview before building	Separated design exploration from build so the agent got one clear target
Data and hosting	Accounts and a backend; static and on-device	Static site on Cloudflare Pages, all data on the device, speech recognised only on the device	No children's data is ever collected, downloads stay tiny and fast, and sharing costs close to nothing

Early signal and success metrics

Early parent feedback is enthusiastic, and the engineering quality bars are met. The learning outcomes still need a playtest with children.

> "This app is amazing and super cute." — Parent of a kindergartner
>
> "This is very cool. Fun little tool." — Parent of a second grader

Because the app collects no analytics by design, success will be measured in moderated playtests and through the on-device history in the grown-ups view, never through tracking.

Metric	Target	Status
Correct strokes accepted by the matcher	99%+ on child-like traces	Met: 99.9–100% in synthetic tests
Reversed strokes caught	95%+	Met: 98%
Drawing frame rate under 4× CPU throttle	40 fps minimum	Met: 50–60 fps
Automated tests	All pass in CI	Met: 76 pass (10 matcher, 24 unit, 42 browser)
Child finishes their name unaided after the grown-up enters it	8 of 10 children	First playtest pending
Child chooses to practise again within a week	6 of 10 families	First playtest pending
Phase-3 stars rise across sessions for the same word	Upward trend over 3 sessions	First playtest pending

What's next

The next bet is evidence, not features. A playtest with children decides what gets built after it.

1. **Harden.** Confirm the offline install and the 150 KB first-load budget on a mid-tier Android phone and a base iPad.
2. **First playtest.** Run five to ten children from the immersion program through their own names. Tune matcher tolerances from real traces, and measure the targets above.
3. **Teacher review.** Have a Japanese-speaking teacher review the 176-name seed list and the voice clips.
4. **Voice input everywhere.** Saying a name instead of typing it already works on Android and Windows, where the browser recognises speech on the device. Next, bring it to iPhone and iPad without sending any audio off the device.

5. **Smarter name spelling.** Add the on-device e2k model as a fallback for names not in the list (4.4 MB, loaded only when needed). The grown-up check stays.
6. **Motivation features.** Ways to make progress visible and give a reason to come back:
   - Hot/cold ink that grows more saturated the closer a stroke is to the model
   - An overall experience tracker that grows with every letter practised
   - Practice goals, such as a name a week or a letter a day
   - Improvement tracking that shows each word's and letter's stars rising over time
   - A reduced-motion mode
7. **Katakana mode.** Katakana is how Japanese normally writes foreign names, and the natural second script once hiragana lands.

What this shows

Each product-lead skill below is tied to a specific call from this project.

Skill	Evidence from this project
AI-native building	Wrote a design doc and agent brief with rules, tolerances and acceptance checks; directed Claude Code through 40+ feedback rounds to a live app in about 10 hands-on hours
Problem framing	Reframed “a kana app” as joining sound and stroke order for pre-readers, with the parent as a separate user
Spec and quality bars	Set measurable bars (40 fps floor, matcher acceptance rates, 150 KB budget) before code existed, and kept them in CI
Scope and trade-offs	Rejected a 1 GB voice download for a 1.8 MB clip set; deferred the name model after it failed 1 in 4 names
Child-centred UX	No failure states, a three-miss fallback to “follow me”, picture-only controls, and voice prompts that take turns instead of overlapping

Skill	Evidence from this project
Design craft	Ran an AI design pipeline: Gemini concepts and mascot → Figma Make tokens and layout → structured plan → Claude Code build; then point-level polish
Audio direction	Picked voices and per-letter takes by ear, and diagnosed clipped clips and wrong vowels letter by letter
Privacy by design	No accounts, analytics or network calls; voice input only where recognition runs on the device; no children's data to protect
Release management	Feature branches, 3 reviewed pull requests, and a Cloudflare Pages deploy for live review before merging

The iteration log, branch status, specs and acceptance checks are in Appendix.